



Modelli di parallelismo e scalabilità

Principi della Programmazione Parallela
CdL Informatica - A.A. 2025/2026



Università
di Catania



INAF
ISTITUTO NAZIONALE
DI ASTROFISICA

Nelle puntate precedenti...

Nelle lezioni precedenti abbiamo visto che:

- Le prestazioni moderne derivano dal **parallelismo**, non dall'aumento della frequenza (fine della “free lunch”).
- L'architettura hardware impone vincoli:
 - multicore
 - gerarchia di memoria
 - memory wall
- Un sistema HPC è una risorsa:
 - condivisa
 - gestita da uno scheduler
 - con ambiente software esplicito

Oggi formalizziamo i **modelli di parallelismo** e il concetto di **scalabilità**.

Obiettivi della lezione

- Distinguere i principali **modelli di parallelismo**
 - data parallelism
 - task parallelism
 - shared vs distributed memory
 - Comprendere il concetto di **scalabilità**
 - Calcolare e interpretare:
 - speedup
 - efficienza
 - Distinguere tra:
 - **strong scaling**
 - **weak scaling**
 - Comprendere i limiti teorici:
 - **Legge di Amdahl**
 - **Legge di Gustafson**
-

Perché parlare di modelli di parallelismo

Un modello di parallelismo:

- definisce **come** il lavoro viene suddiviso
- determina **come** le unità di esecuzione interagiscono
- implica un certo **modello di memoria**
- introduce specifici costi di:
 - comunicazione
 - sincronizzazione
 - gestione dei dati

Il modello scelto influenza:

- la progettazione dell'algoritmo
 - la struttura del codice
 - la scalabilità del programma
-

Che cos'è il parallelismo

Esecuzione simultanea di più unità di lavoro al fine di ridurre il tempo di esecuzione o aumentare la dimensione del problema risolvibile.

Concurrency

- Più attività in progresso logico
- Non necessariamente simultanee
- Può avvenire anche su un singolo core

Parallelism

- Esecuzione fisicamente simultanea
 - Richiede più risorse hardware
-

Modelli di parallelismo (panoramica)

Come si suddivide il lavoro

Data Parallelism

- Stessa operazione applicata a porzioni diverse di dati
- Tipico di:
 - a. loop numerici
 - b. simulazioni scientifiche
- Dominante in HPC scientifico

Task Parallelism

- Compiti diversi eseguiti in parallelo
 - Workflow, pipeline, servizi indipendenti
 - Tipico in applicazioni eterogenee
-

Modelli di parallelismo (panoramica)

Come è organizzata la memoria

Shared memory

- Spazio di indirizzamento unico
- Comunicazione implicita
- Scalabilità limitata al nodo
-

Distributed Memory

- Memoria privata per processo
- Comunicazione esplicita
- Scalabilità su cluster

I sistemi HPC moderni sono spesso **ibridi**.

Memoria condivisa (Shared Memory)

Nel modello a memoria condivisa:

- Tutte le unità di esecuzione condividono uno **spazio di indirizzamento globale**
- Le variabili possono essere accessibili da più thread
- La comunicazione avviene:
 - implicitamente
 - tramite accesso a memoria condivisa

Caratteristiche principali

- Programmazione relativamente intuitiva
- Bassa latenza di comunicazione intra-nodo
- Richiede meccanismi di sincronizzazione:
 - lock
 - barrier
 - sezioni critiche

Problemi tipici

- Race condition
- Data corruption
- False sharing
- Scalabilità limitata alla dimensione del nodo

Memoria distribuita (Distributed Memory)

Nel modello a memoria distribuita:

- Ogni processo ha **memoria privata**
- Non esiste uno spazio di indirizzamento globale
- I dati devono essere **scambiati esplicitamente**

Caratteristiche principali

- Comunicazione tramite:
 - messaggi espliciti
 - scambio di dati tra processi
- Maggiore controllo sulla distribuzione dei dati
- Maggiore complessità di programmazione

Vantaggi

- Elevata scalabilità
 - Adatto a sistemi multi-nodo
 - Modello naturale per cluster HPC
-

Strong scaling vs Weak scaling

Esistono due modalità fondamentali di analisi della scalabilità.

Strong Scaling

- Problema **di dimensione fissa**
- Si aumenta il numero di processi p
- Si misura la riduzione del tempo

$$T(p) \downarrow \quad \text{con problema costante}$$

Ideale

$$T(p) = \frac{T(1)}{p}$$

Weak Scaling

- Il problema cresce con p
- Il carico per processo resta costante
- Si misura la variazione del tempo

$$T(p) \approx \text{costante}$$

Scalabilità - concetti fondamentali

La **scalabilità** è la capacità di un sistema parallelo di migliorare le prestazioni al crescere delle risorse utilizzate.

Tempo di esecuzione

- $T(1)$ → tempo con 1 processo
- $T(p)$ → tempo con p processi

Speedup

$$S(p) = \frac{T(1)}{T(p)}$$

Efficienza

$$E(p) = \frac{S(p)}{p}$$

Ideale:

- $S(p) = p$
- $E(p) = 1$

In pratica:

- $S(p) < p$
 - $E(p) < 1$
-

Perchè lo speedup non è lineare

Lo speedup lineare $S(p) = p$ è un caso ideale. Nella pratica intervengono diversi fattori:

- **Parte seriale**
 - Una frazione del programma non è parallelizzabile
- **Comunicazione**
 - Scambio di dati tra processi, Latenza e banda di rete
- **Sincronizzazione**
 - Barrier, Sezioni critiche, Attese tra thread/processi
- **Overhead di gestione**
 - Creazione thread/processi, Scheduling, Runtime
- **Squilibrio di carico (Load imbalance)**
 - Alcuni processi terminano prima di altri

Lo speedup reale è sempre inferiore allo speedup ideale

Legge di Amdahl

Sia:

- α La frazione parallelizzabile del programma
- $1 - \alpha$ = parte seriale

Il tempo di esecuzione con p processi è $T(p) = T(1) \left[(1 - \alpha) + \frac{\alpha}{p} \right]$

Lo speedup è $S(p) = \frac{1}{(1 - \alpha) + \frac{\alpha}{p}}$

Limite teorico $\lim_{p \rightarrow \infty} S(p) = \frac{1}{1 - \alpha}$

Anche con infiniti processi, la parte seriale impone un limite allo speedup.

Mini esercizio: Legge di Amdahl

Supponiamo che:

- Il 95% del programma sia parallelizzabile $\alpha = 0.95$
- Il restante 5% sia seriale $1 - \alpha = 0.05$

Domande

1. Qual è lo speedup massimo teorico?
 2. Qual è lo speedup con:
 - $p = 4$ processi?
 - $p = 16$ processi?
-

Mini esercizio: Legge di Amdahl

Limite teorico

$$\lim_{p \rightarrow \infty} S(p) = \frac{1}{1 - \alpha}$$

$$S_{max} = \frac{1}{0.05} = 20$$

Anche con infiniti processi, lo speedup non supera 20

Mini esercizio: Legge di Amdahl

Caso 1 – $p = 4$

$$S(4) = \frac{1}{0.05 + \frac{0.95}{4}} = \frac{1}{0.05 + 0.2375} = \frac{1}{0.2875} \approx 3.48$$

Mini esercizio: Legge di Amdahl

Caso 2 – $p = 16$

$$S(16) = \frac{1}{0.05 + \frac{0.95}{16}} = \frac{1}{0.05 + 0.059375} = \frac{1}{0.109375} \approx 9.14$$

Mini esercizio: Legge di Amdahl

Supponiamo che:

- Il 10% del programma sia seriale $1 - \alpha = 0.1$

Domanda

È possibile ottenere:

- Speedup **50x** ?
 - Speedup **20x** ?
 - Speedup **5x** ?
-

Legge di Gustafson

La Legge di Gustafson considera il caso di **weak scaling**:

- Il problema cresce con il numero di processi p
- La parte seriale resta costante
- La parte parallelizzabile cresce con p

Lo speedup diventa: $S(p) = p - (1 - \alpha)(p - 1)$

Interpretazione

- Se α è vicino a 1, lo speedup cresce quasi linearmente
- Non esiste un limite rigido come in Amdahl nel caso di problema crescente

Gustafson descrive meglio la pratica HPC reale

Scalabilità reale vs scalabilità teorica

Le leggi di Amdahl e Gustafson descrivono limiti **ideali**.

Nella pratica intervengono ulteriori fattori:

- **Comunicazione**
 - Latenza di rete, Banda disponibile, Contesa tra processi
- **Gerarchia di memoria**
 - Cache, NUMA, Accessi non uniformi
- **Sincronizzazione**
 - Barrier, Lock, Attese
- **Squilibrio di carico**
 - Partizionamento non uniforme, Dinamica del problema

La scalabilità reale è sempre inferiore al limite teorico

Quando ha senso aggiungere processi?

Aumentare il numero di processi è vantaggioso quando:

- **La parte seriale è minima**
 - $1 - \alpha$ molto piccolo, Limite teorico elevato
- **Il problema è sufficientemente grande**
 - Granularità adeguata, Il tempo di calcolo domina la comunicazione
- **La comunicazione non domina il tempo totale**
 - Bassa latenza, Buona banda disponibile
- **Il carico è ben bilanciato**
 - Partizionamento uniforme, Assenza di colli di bottiglia

Oltre un certo punto, aggiungere processi può:

- non migliorare le prestazioni
 - peggiorare l'efficienza
 - aumentare il costo computazionale
-

Messaggi chiave della lezione

- Non esiste un solo parallelismo, ma diversi **modelli**.
- Il modello scelto determina:
 - organizzazione del lavoro
 - modello di memoria
 - modalità di comunicazione
- La scalabilità si misura tramite:
 - **speedup**
 - **efficienza**
- Strong scaling e weak scaling descrivono scenari diversi.
- La **Legge di Amdahl** impone un limite allo strong scaling.
- La **Legge di Gustafson** descrive la scalabilità nei problemi crescenti.
- La scalabilità reale è sempre inferiore a quella teorica.

Programmare in parallelo significa progettare per la scalabilità.
